2026 International Conference on Intelligent Processing, Hardware, Electronics, and Radio Systems (CIPHER) | 979-8-3315-5184-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/CIPHER70417.2026.11523913

Design,Implementation,and Analysis of 64-bit Hybrid Adders Using Carry Look-Ahead and Carry Select Techniques on FPGA

1st Gouthami Valigonda,
M.Tech in Embedded System Design
National Institue of Technology, Kurukshetra
Haryana, India
gouthamivaligonda@gmail.com

2nd R.K.Sharma
Electronics and Communication Engineering
National Institue of Technology, Kurukshetra
Haryana, India
rksharma@nitkkr.ac.in

***Abstract*—This project focuses on the design, implementation, and analysis of various 64-bit adder architectures using Verilog, targeting the Xilinx Artix-7 FPGA. Three distinct types of adders were developed: a conventional 64-bit Ripple Carry Adder (RCA), a 64-bit Carry Look ahead Ripple carry adder composed of sixteen 4-bit Carry Look-Ahead Adder (CLA) blocks connected in a ripple fashion, and a 64-bit Carry Select Adder (CSA) made of sixteen blocks, each containing two 4-bit CLA units to handle carry-in values of 0 and 1. The primary goal was to explore architectural differences and validate the functional correctness of each design. All adders were implemented on the FPGA and interfaced with VIO (Virtual Input/Output) to apply inputs dynamically. The Integrated Logic Analyzer (ILA) was used to observe the output of individual 4-bit segments. The analysis phase focused on verifying that the final 64-bit sum output was accurate for all test inputs. It was observed that all three adder architectures produced the same correct output values for every test case, demonstrating functional consistency across designs. This project highlights the modular design of arithmetic circuits and the effective use of FPGA debugging tools for functional verification.**

***Index Terms*—Full Adder(FA), Carry Look Ahead Adder (CLA), Carry Select Adder(CSA), Field Programmable Gate Array (FPGA), Virtual Input/Output (VI0), Intergrated Logic Analyzer (ILA)**

I. Introduction

Efficient arithmetic operations are fundamental to the performance of modern digital systems, especially in processors and high-speed computing units. Among these, the addition operation plays a crucial role and is often a key factor in determining the speed and resource utilization of a design. With the growing demand for high-performance computing, exploring different adder architectures and understanding their trade-offs has become essential. Field- Programmable Gate Arrays (FPGAs) offer a flexible platform for prototyping and evaluating such digital designs due to their reconfigurability and integrated debugging tools. This project leverages the capabilities of the Xilinx Artix-7 FPGA to design, implement, and analyze various 64-bit adder architectures using Verilog, focusing on both structural differences and functional correctness.

A. Importance of the Adders in ALUs

Adders are essential components of an Arithmetic Logic Unit (ALU) because they perform the fundamental arithmetic operations required in all digital systems. They enable basic addition and support subtraction using two's complement logic. Since many complex operations like multiplication and division rely on repeated addition and subtraction, the efficiency of an adder directly affects overall processor performance. As adders often lie on the ALU's critical path, their speed significantly influences instruction execution time. Modern processors also require adders that can efficiently handle larger bit-widths, motivating the use of advanced architectures such as Carry Look-Ahead and Carry Select Adders. Beyond arithmetic, adders are vital for tasks like address calculation, program counter updates, and branch processing. Because of their central role, adders serve as key targets for design optimization, balancing speed, area, and power in digital system design.

B. Virtual input Output and Integrated Logic Analyzer

In FPGA-based system design, debugging and signal observation play a critical role in verifying and validating the functionality of digital circuits. Xilinx Vivado provides two important IP cores for on-chip debugging: VIO (Virtual Input/Output) and ILA (Integrated Logic Analyzer). These cores allow designers to monitor and interact with internal signals in real-time, without requiring additional physical interfaces.
The VIO core enables real-time monitoring and driving of internal FPGA signals, allowing users to apply inputs and observe outputs dynamically during runtime. This is especially useful for controlling and testing parts of the design on-the-fly.
The ILA core, on the other hand, captures and displays internal signal activity based on user- defined trigger conditions. It functions as a logic analyzer embedded within the FPGA,providing detailed waveform data that helps identify timing issues, logic errors, or protocol mismatches.
Together, VIO and ILA provide a powerful platform for efficient and effective debugging, reducing design cycles and improving design visibility during development and testing.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.

II. DESIGN AND METHODLOGY

A. Different Types Of Adders

1) Ripple Carry Adder: Ripple Carry adder is designed using Full adders which generates sum of two arithmetic binary integers. The addition is done bit by bit starting from Least Signficant Bit (LSB) to Most Signficant bit (MSB).A full adder is a digital combinational circuit that generates an output of 2 bits (sum, carry) after forming the arithmetic sum of 3 input bits namely p, q and r, where p and q are bits to be added, and r is the input carry.The two outputs can be calculated using

Sum $= p \oplus q \oplus r$

Carry $= p.q + p.r + q.r$

An n-bit Ripple Carry Adder (RCA) is implemented using n cascaded full adders, where the carry output of each stage serves as the carry input for the subsequent stage. This sequential dependency requires each full adder to wait for the propagation of the carry from its predecessor before generating its own sum and carry outputs. Consequently, the overall delay of the RCA increases linearly with the number of bits, as the carry signal ripples through all stages.

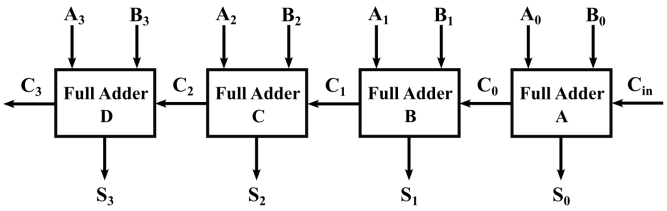


Fig. 1. 4-bit Ripple Carry Adder

2) Carry Look Ahead Adder: A carry look ahead adder is also known as fast adders,as these adders increase speed by decreasing the time needed to create carry bits. The CLA logic decides whether each bit pair in the binary sequence that needs to be added generates a carry or propagates a carry. The following logic's can be used to derive the carry propagate (Pi) and carry generate (Gi) functions

$P_i = A_i \oplus B_i$ and $G_i = A_i.B_i$

By using these equations, we can generate an output sum and carry functions

Sum $= P_i \oplus C_i$ and $C_{i+1} = P_i.C_i + G_i$

Irrespective of Carry Input, the Carry generate G_i produces Carry out as 1, if $A_i and B_i$ are 1. If a carry enters stage i, The Carry Propagate P_i decides whether it advances to i+1 stage.

3) Carry Select adder: A Carry Select Adder (CSA) is a hardware circuit designed to add two binary numbers faster than a conventional ripple carry adder. The key idea of a CSA is to precompute the sum and carry for each block twice: once assuming the carry-in is 0 and once assuming the carry-in is 1. When the actual carry-in becomes available, the correct precomputed sum and carry are selected using multiplexers. This eliminates the long wait for carry propagation across all bits, reducing overall addition delay.

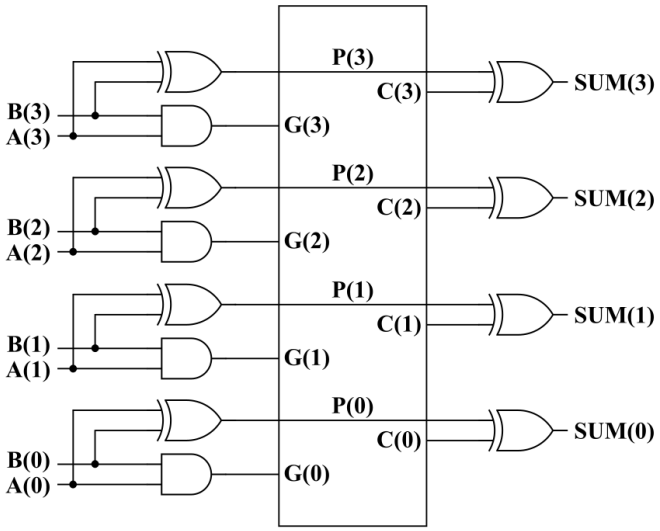


Fig. 2. 4-bit Carry Look Ahead Adder

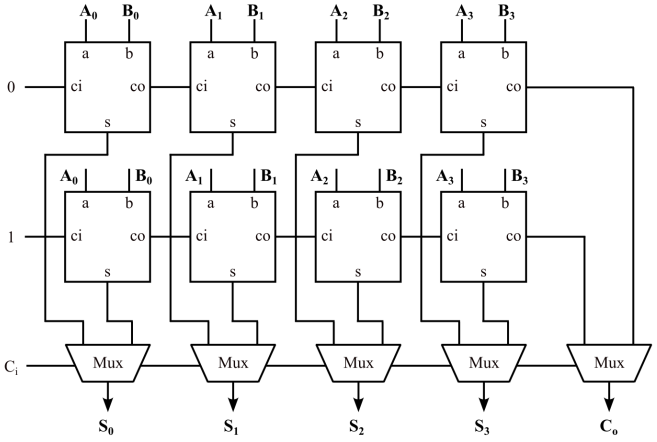


Fig. 3. 4-bit Carry Select Adder

B. Implementation Of 64-bit Ripple Carry Adder

A 64-bit Ripple Carry Adder (RCA) is a hardware circuit designed to add two 64-bit binary numbers. The ripple carry adder is one of the simplest forms of adder and uses full adders to perform the addition in a sequential manner.

1) Structure:

Inputs:

- Two 64-bit binary numbers, $A = A_{63}A_{62}\ldots A_1A_0$ and
  $B = B_{63}B_{62}\ldots B_1B_0$
  Each bit is represented by A_i and B_i
  where $i = 0, 1, \ldots, 62, 63$.

Output:

- A 64-bit sum, $S = S_{63}S_{62}\ldots S_1S_0$, where S_0 is the least significant bit (LSB) and S_{63} is the most significant bit (MSB).
- C_{out} is the carry out.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.

2) Operation:

- **Full Adder Units:**
  - Each bit pair from A and B is added using a full adder:
    $S_i = A_i \oplus B_i \oplus C_{i-1}$
    - $\oplus$ denotes the XOR operation (to calculate the sum bit).
    - C_{i-1} is the carry-in to the ith full adder, which is the carry-out from the previous stage.
- **Carry Calculation:**
  - The carry-in to each full adder is propagated from the carry-out of the previous stage: $C_i = (A_i \cdot B_i) + (A_i \cdot C_{i-1}) + (B_i \cdot C_{i-1})$
  - This equation calculates the carry-out from one bit to the next.
- **Sequential Processing:**
  - The carry bit C_0 (initial carry-in) starts with a value of 0.
  - The output carry C_{out} is the final carry-out after the addition of all bits.

3) Delay Analysis:

- **Critical Path:** The delay through a 16-bit RCA is determined by the longest propagation delay through any full adder in the sequence.
- **Adder Delay:** Each full adder consists of three logic gates (XOR, AND, OR) and thus has a certain propagation delay.
- **Ripple Delay:** Since the carry-out of one stage is the carry-in for the next, this leads to a ripple effect, hence the name *ripple carry adder*.
- **RCA Delay Equation:** The delay of an n-bit Ripple Carry Adder (RCA) can be expressed as:
  $T_{\text{RCA}} = (N-1) \times \text{Carry Delay} + \text{Full Adder Sum Delay}$.

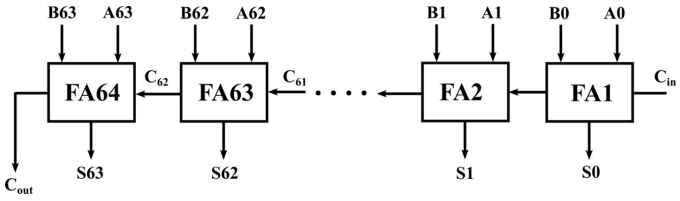


Fig. 4. 64-bit Ripple Carry Adder

C. 64-bit Carry Look Ahead Ripple Carry Adder

A 64-bit Carry Lookahead Ripple Carry Adder (CLA-RCA) is a hardware circuit designed to add two 64-bit binary numbers. In this design, the adder is divided into 16 blocks of 4-bit Carry Lookahead Adders (CLA).The blocks are then connected in a ripple-carry fashion, where the carry-out of one block serves as the carry-in of the next block.

1) Structure:

- **Inputs:**
  - Two 64-bit binary numbers,
    $A = A_{63}A_{62}\ldots A_1A_0$ and
    $B = B_{63}B_{62}\ldots B_1B_0$
  - Each bit is represented by A_i and B_i, where $i = 0, 1, \ldots, 62, 63$.
- **Output:**
  - A 64-bit sum,
    $S = S_{63}S_{62}\ldots S_1S_0$, where S_0 is the least significant bit (LSB) and S_{63} is the most significant bit (MSB).
  - C_{out} is the carry-out.

2) Operation:

- **4-bit Carry Lookahead Adder Blocks:**
  - The 64-bit adder is divided into 16 blocks of 4-bit Carry Lookahead Adders (CLA).
  - Each 4-bit CLA block quickly computes the sum and carry for its 4 bits using lookahead logic, which predicts carry bits faster than ripple carry.
- **Carry Propagation Between Blocks:**
  - The carry-out of each 4-bit CLA block is connected as the carry-in of the next block in a ripple carry fashion.
  - This means the carry ripples between blocks, but inside each block, carry generation is fast due to carry lookahead logic.
- **Sequential Block Processing:**
  - The initial carry-in (C_0) is zero.
  - Sum and carry outputs from each block depend on the carry propagated from the previous block.

3) Delay Analysis:

- **Block Delay:** Each 4-bit CLA block generates carries internally with reduced delay compared to ripple carry.
- **Ripple Delay Between Blocks:** Since carry propagates sequentially between 16 blocks, overall delay depends on ripple carry delay across the blocks.
- **Total Delay:** The delay is significantly improved compared to a 64-bit ripple carry adder but still affected by the ripple delay across blocks.
- **CLARCA Delay Equation:** The delay of an n-bit CLA-RCA can be expressed as:
  $T_{\text{CLA-RCA}} = (N/4 - 1) \times$ Carry Delay of CLA + CLA Sum Delay

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.

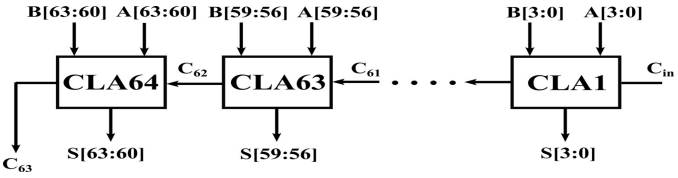


Fig. 5. 64-bit Carry Look Ahead Ripple Carry Adder

D. 64-bit Carry Select Adder using 4-bit Carry Look Ahead Adder blocks

A 64-bit Carry Select Adder Using 4-Bit Carry Lookahead Blocks is a hardware circuit designed to add two 64-bit binary numbers efficiently. The adder is divided into 16 stages, with each stage consisting of two 4-bit Carry Lookahead Adder (CLA) blocks. One CLA in each stage computes the sum and carry assuming the carry-in is 0, while the other computes the sum and carry assuming the carry-in is 1. Once the actual carry-in from the previous stage is available, a multiplexer selects the correct sum and carry output from the two precomputed sets. The stages are connected in a carry select fashion, allowing most of the addition to be computed in parallel and significantly reducing the carry propagation delay compared to a conventional ripple carry adder.

1) Structure:

- **Inputs:**
  - Two 64-bit binary numbers, $A = A_{63}A_{62}\ldots A_1A_0$ and $B = B_{63}B_{62}\ldots B_1B_0$
  - Each bit is represented by A_i and B_i, where $i = 0, 1, \ldots, 62, 63$.
- **Output:**
  - A 64-bit sum, $S = S_{63}S_{62}\ldots S_1S_0$
  - C_{out} is the carry-out

2) Operation:

- **4-bit Carry Lookahead Adder Blocks:**
  - The 64-bit input is divided into 16 blocks of 4-bit CLAs.
  - Each 4-bit block computes two sums and two carry outputs in parallel — one assuming carry-in is 0, the other assuming carry-in is 1.
- **Carry Select Mechanism:**
  - Once the actual carry-in for the block is known, a multiplexer selects the correct sum and carry output from the two precomputed sets.
  - This eliminates the need to wait for the carry to ripple through each bit or block sequentially.
- **Parallel Computation:**
  - By calculating sums for both possible carry-in values in parallel, carry propagation delay is reduced drastically.

3) Delay Analysis:

- Delay is significantly reduced compared to a standard ripple carry adder.
- Within each 4-bit block, carry lookahead logic ensures fast computation.
- The overall addition time is determined by the multiplexer selection delay and the longest carry propagation between blocks.
- **CLA-CSA Delay Equation:** The delay of an 64-bit CSA Using 4-bit CLA can be expressed as: $T_{\text{CSA-CLA}}$ = Approximately 4bit-CLA adder delay $+ 15\times$ 2 to 1 MUX Delay.

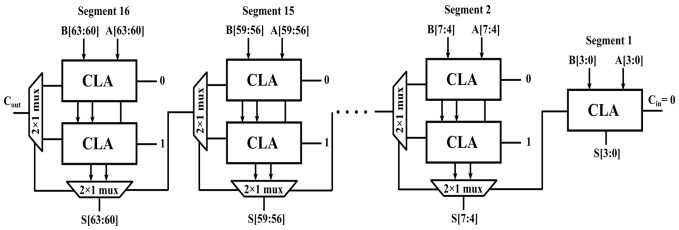


Fig. 6. 64-bit CSA using 4-bit CLA Adder blocks

E. FPGA Implementation and Verification Using VIO and ILA

The adder designs were implemented on the Xilinx Artix-7 FPGA using the Vivado Design Suite. The Artix-7 FPGA provides a balance of performance and resource availability suitable for evaluating different adder architectures.

1) Procedure:

- **Creating the Block Design:**
  - Open Vivado and create a new project targeting the Artix-7 FPGA device.
  - Use the IP Integrator to create a new block design.
  - Add the custom adder modules — the 64-bit Ripple Carry Adder, the Hierarchical 64-bit Carry Lookahead Ripple Carry Adder, and the 64-bit Carry Select Adder using 4-bit CLA blocks — as IP blocks in the design.
  - Include VIO (Virtual Input/Output) and ILA (Integrated Logic Analyzer) IP cores for real-time monitoring and debugging.
- **Mapping Ports:**
  - Connect the input ports of each adder module (two 64-bit operands and carry-in) to the VIO core inputs, enabling dynamic input from the user during testing.
  - Connect the output ports (sum and carry-out) of the adders to the ILA core inputs to capture and analyze internal signals and results.
  - Properly map clock and reset signals required by the adder modules and debug IP cores.
  - Validate the block design for connectivity and correctness.
- **Generating HDL Wrapper and Exporting Design:**
  - After completing the block connections, generate the HDL wrapper file for the block design.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.

  - Run synthesis and implementation to prepare the design for FPGA programming.
  - Generate the bitstream file (.bit) to be loaded onto the FPGA.
- **Programming and Verification:**
  - Program the Artix-7 FPGA with the generated bitstream using Vivado Hardware Manager.
  - Use the VIO core to apply different input test vectors dynamically without reprogramming the FPGA.
  - The ILA core captures output responses and internal signals based on trigger conditions set in Vivado.
  - Monitor the captured waveforms in Vivado's Logic Analyzer to verify the correctness of adder operations for various input combinations.

III. RESULTS AND SIMULATION

A. Power and Area Analysis

The power and Area analysis of three different architectures are shown below in Table 1,2,3 and Figure 7,8,9 respectively.

TABLE I
AREA ANALYSIS OF 64-BIT RIPPLE CARRY ADDER

Resource	Utilization	Available	Utilization (%)
LUT	2067	63400	3.26
LUTRAM	286	19000	1.51
FF	3638	1268000	2.87
BRAM	5.50	135	4.07
IO	01	210	0.48
BUFG	02	32	6.25

TABLE II
AREA ANALYSIS OF 64-BIT CARRY LOOKAHEAD RIPPLE CARRY ADDER

Resource	Utilization	Available	Utilization (%)
LUT	2067	63400	3.26
LUTRAM	286	19000	1.51
FF	3638	1268000	2.87
BRAM	5.50	135	4.07
IO	01	210	0.48
BUFG	02	32	6.25

TABLE III
AREA ANALYSIS OF 64-BIT CSA USING 4-BIT CLA BLOCKS

Resource	Utilization	Available	Utilization (%)
LUT	2080	63400	3.28
LUTRAM	286	19000	1.51
FF	3638	1268000	2.87
BRAM	5.50	135	4.07
IO	01	210	0.48
BUFG	02	32	6.25

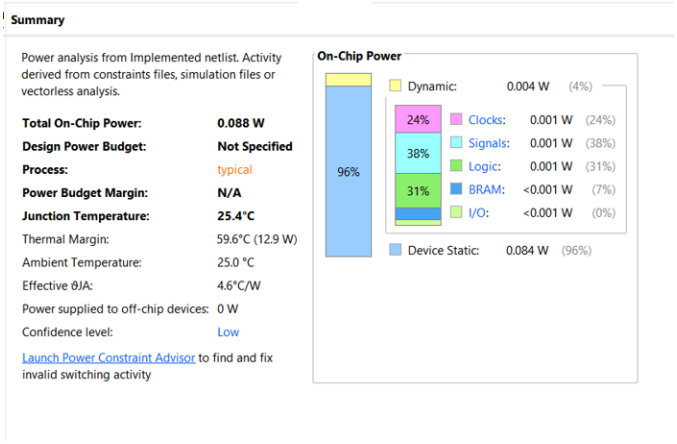


Fig. 7. Power Analysis of 64-bit Ripple Carry Adder

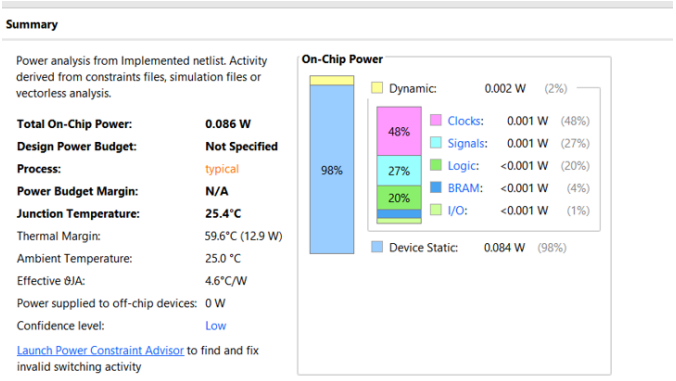


Fig. 8. Power Analysis of 64-bit CLA Ripple Carry Adder

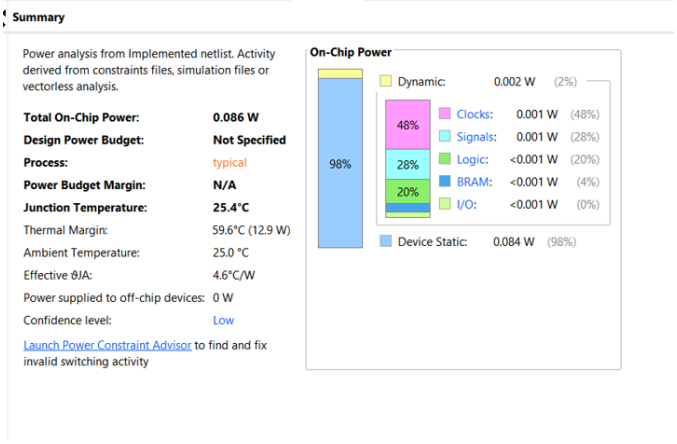


Fig. 9. Power Analysis of 64-bit CSA using 4-bit CLA blocks

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.

B. Simulation Results

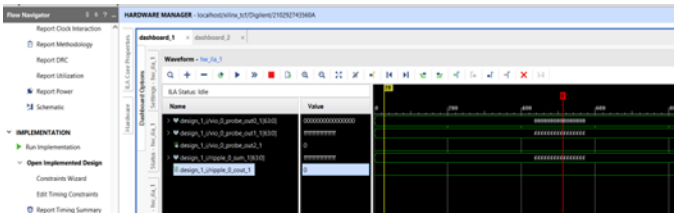


Fig. 10. Simulation results of 64-bit ripple carry adder in ILA

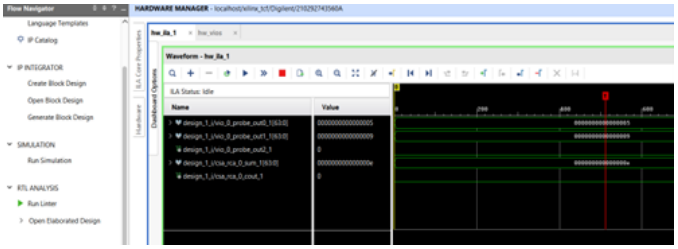


Fig. 11. Simulation results of 64-bit CLA RCA in ILA

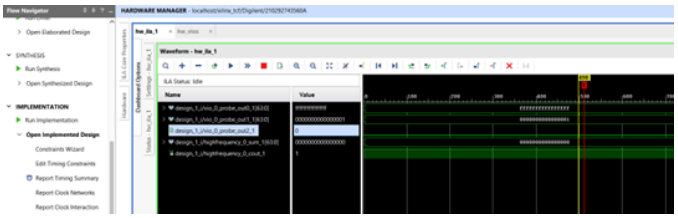


Fig. 12. Simulation results of 64-bit CSA using 4-bit CLA blocks in VIO

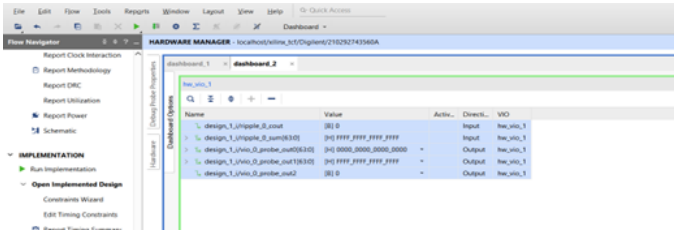


Fig. 13. Simulation results of 64-bit ripple carry adder in VIO

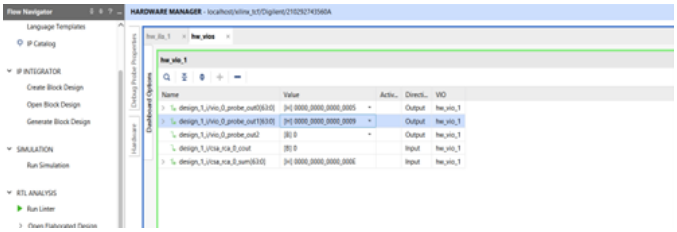


Fig. 14. Simulation results of 64-bit CLA RCA in VIO

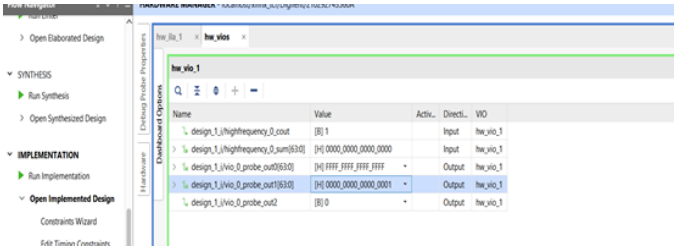


Fig. 15. Simulation results of 64-bit CSA using 4-bit CLA blocks in ILA.

IV. Conclusion and Future Directions

This project explored the design, implementation, and analysis of three different 64-bit adder architectures: the Ripple Carry Adder (RCA), the hybrid Carry Look-Ahead (CLA) based ripple structure, and the Carry Select Adder (CSA) using 4-bit CLA blocks, all implemented on the Xilinx Artix7 FPGA. Each adder was modularly designed in Verilog and verified using FPGA debugging tools such as the Virtual Input/Output (VIO) and Integrated Logic Analyzer (ILA).

The Ripple Carry Adder was simple and easy to implement but suffered from significant propagation delays due to sequential carry transfer, limiting its speed for wide-bit additions. The hybrid CLA-based ripple structure improved speed within small 4-bit blocks, providing a balance between performance and resource utilization. The Carry Select Adder, which uses two 4-bit CLA blocks per stage to precompute sums for both possible carry-in values and select the correct result via multiplexers, was designed to reduce carry propagation delays and exploit parallelism.

Interestingly, the resource utilization (LUTs, flip-flops, and carry-chain usage) was similar across all three architectures. This is because all designs ultimately implement the same 64-bit addition logic; the differences in internal structure (carry ripple, CLA blocks, or precomputed CSA blocks) are mapped efficiently to the FPGA's LUTs and dedicated carry-chain hardware by the synthesis tools, which optimize for minimal resource usage. Consequently, while the architectures differ in conceptual design, the FPGA implementation uses roughly the same number of logic resources.

Timing analysis highlighted the performance differences between the architectures:The RCA, due to sequential carry propagation, had the slowest critical path (11.8 ns) and a maximum frequency of 84.7 MHz.

The hybrid CLA ripple adder achieved the fastest timing, with a critical path of 6.82 ns and a maximum frequency

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.

of 146.6 MHz, leveraging the FPGA's dedicated carry-chain logic.

The CSA had a critical path of 8.97 ns and a maximum frequency of 111.5 MHz; although CSA precomputes sums in parallel, the additional multiplexer delays and LUT usage reduced its practical speed advantage on FPGA.

All three adders were functionally correct across various test inputs, confirming the reliability of the FPGA-based implementations. This study demonstrates that FPGA-specific hardware characteristics, such as dedicated carry chains, can influence which adder architecture achieves higher performance, even when theoretical analysis might suggest otherwise.

Future work can include implementing these adders on different FPGA families, such as Zynq, Spartan, or Intel FPGAs, to explore cross-platform performance and resource usage. Additionally, formal verification techniques, including exhaustive input testing and sequential equivalence checking, could be applied to ensure functional correctness across all architectures

REFERENCES

[1] J.Perika Puneeth, Ajay, Sonal Yadav 2023 Design and Implementation of High Frequency 16-bit full adder on FPGA Families.4th International Conference for Emerging Technology (INCET). IEEE 2023

[2] Surekha, Gajula Madesh, Pavan Kumar, Hrushikesh 2023 Design and Implementation of Arithmetic and Logic Unit 2nd International Conference Applied Artificial Intelligence And Computing (ICAAIC). IEEE 2023.

[3] Yu-Ting Pai and Yu-Kumg Chen 2004 The Fastest Carry Look Ahead Adder 2nd IEEE International Workshop on Electronic Design, Test and Applications (DELTA'04)

[4] R. V. Nithyashree, S. Afreen and S. Tantry, "Analysis of various Approximate adders in Ripple Carry Adder design.," 2023 4th IEEE Global Conference for Advancement in Technology (GCAT), Bangalore, India, 2023.

[5] M. Hasan, P. Biswas, M. S. Alam, H. U. Zaman, M. Hossain and S. Islam, "High Speed and Ultra Low Power Design of Carry-Out Bit of 4-Bit Carry Look-Ahead Adder," 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT), Kanpur, India, 2019.

[6] Samir palnitkar, "VERILOG HDL", a guide to digital design and synthesis IEEE 1364- 2001 complaint,2nd edition.

[7] Digital Logic and Computer Design by M. Morris Mano (2nd Edition).

[8] NPTEL lectures on Hardware Modelling using Verilog.

[9] B. Koyada, N. Meghana, M. O. Jaleel and P. R. Jeripotula, "A comparative study on adders," 2017 International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET), Chennai, India, 2017.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:35:34 UTC from IEEE Xplore. Restrictions apply.